UD 4. Acceso a servicios web

1 Repaso de los métodos básicos HTTP

En Angular (incluida la versión 20), las operaciones HTTP se realizan utilizando el servicio **HttpClient**, parte del módulo HttpClientModule. Este servicio proporciona una API basada en **Observables** que permite consumir APIs REST de forma reactiva, tipada y segura.

Los métodos HTTP más comunes son:

- **GET**: Consulta datos y normalmente no modifica nada en la base de datos. Equivale a SELECT en SQL. Cuando usamos este método, la información normalmente se envía concatenada en la URL (formato URIEncoded).
- **POST**: Operación para insertar un nuevo dato (no necesariamente) en el servidor. Equivale a INSERT en SQL. Los datos a insertar se envían dentro del cuerpo de la petición HTTP.
- **PUT**: Esta operación actualiza datos existentes en el servidor. Equivale a UPDATE en SQL. La información que identifica el objeto a actualizar es enviada en la URL (como en GET), y los datos a modificar se envían aparte en el cuerpo de la llamada (como en POST).
- **DELETE**: Esta operación elimina un dato del servidor, como la operación DELETE de SQL. La información que identifica al objeto a eliminar será enviada en la URL (como en GET).

2 RxJS y observables

Angular utiliza **RxJS** para manejar flujos de datos asíncronos. Cada llamada HTTP, evento y stream en Angular se expresa mediante **Observables**, lo que permite trabajar de forma reactiva, cancelable y componible.

Un **Observable** es una fuente de datos que "emite valores" a lo largo del tiempo. Un componente **se suscribe** para recibir esos datos.

Podríamos compararlo con un **canal de YouTube**:

- El Observable es el canal.
- Suscribirse es pulsar "Suscribirse".
- Cada nuevo vídeo es un valor emitido.
- Puedes dejar de recibir contenido si te **desuscribes**.

Para crear un Observable

```
import { Observable } from 'rxjs';

const miObservable = new Observable<string>(observer => {
  observer.next('Primer mensaje');
  observer.next('Segundo mensaje');
  observer.complete();
});
```

Consumirlo

```
miObservable.subscribe({
  next: valor => console.log(valor),
  complete: () => console.log('Completado')
});
```

Y tendría una salida

```
Primer mensaje
Segundo mensaje
Completado
```

En Angular, las peticiones HTTP se basan en Observables:

```
getUsuarios() {
  return this.http.get<Usuario[]>('https://api.ejemplo.com/usuarios');
}
```

Consumirlo desde un componente:

```
this.servicio.getUsuarios().subscribe(usuarios => {
  console.log('Usuarios:', usuarios);
});
```

Los **operadores** permiten transformar, filtrar o combinar Observables.

Map: Transforma los valores que emite el Observable.

```
import { map } from 'rxjs/operators';

this.api.getUsuarios()
  .pipe(
    map(lista => lista.length) // transforma la lista en su longitud
  )
  .subscribe(total => console.log('Número de usuarios:', total));
```

Filter: Filtra valores.

```
import { filter } from 'rxjs/operators';

of(1, 2, 3, 4, 5)
  .pipe(filter(n => n % 2 === 0))
  .subscribe(n => console.log(n)); // 2, 4
```

Tap: Permite ejecutar side-effects sin modificar el valor.

```
this.api.getUsuarios()
  .pipe(
    tap(() => console.log('Petición enviada')),
  )
  .subscribe();
```

switchMap: Cambia de un Observable a otro (muy útil en búsquedas o dependencias).

```
this.buscador.valueChanges
  .pipe(
    switchMap(texto => this.api.buscarUsuarios(texto))
  )
  .subscribe(resultado => console.log(resultado));
```

catchError: Captura errores en la cadena.

```
import { catchError, of } from 'rxjs';

this.api.getUsuarios()
  .pipe(
    catchError(error => {
      console.error('Error:', error);
      return of([]); // devuelvo un array vacío como fallback
    })
  )
  .subscribe();
```

Cuando te suscribes a un Observable largo (intervalos, eventos, etc.), debes **desuscribirte** para evitar fugas de memoria.

```
import { Subscription } from 'rxjs';
```

```
suscripcion!: Subscription;

ngOnInit() {
  this.suscripcion = this.api.getUsuarios().subscribe();
}

ngOnDestroy() {
  this.suscripcion.unsubscribe();
}
```

3 Procesando respuestas y errores con Observables

Cuando Angular recibe datos a través de HttpClient, lo hace mediante **Observables**.

Esto permite:

- Transformar la respuesta antes de entregarla al componente.
- Controlar errores sin que la app falle.
- Encadenar operadores de RxJS para manipular los datos.

Procesar respuestas con map

map permite transformar la respuesta antes de suscribirse. Como ejemplo sería devolver solo los nombres de los usuarios

```
getNombresUsuarios() {
                                                                return
this.http.get<Usuario[]>('https://api.ejemplo.com/usuarios')
    .pipe(
      map(usuarios => usuarios.map(u => u.nombre))
    );
}

Consumición:

this.api.getNombresUsuarios().subscribe(nombres => {
  console.log(nombres);
});
```

Procesar respuestas con tap (side effects)

tap NO modifica la respuesta. Sirve para registrar, mostrar logs o ejecutar acciones secundarias.

```
getProductos() {
  return
this.http.get<Producto[]>('https://api.ejemplo.com/productos')
    .pipe(
      tap(() => console.log('Respuesta recibida del servidor')),
      tap(productos => console.log('Total:', productos.length))
    );
}
```

Procesar errores con catchError

Cuando ocurre un error HTTP (404, 500, timeout…), Angular lanza un error en el Observable. Lo capturas así:

```
import { catchError, of } from 'rxjs';

getUsuarios() {
  return
this.http.get<Usuario[]>('https://api.ejemplo.com/usuarios')
    .pipe(
      catchError(error => {
        console.error('Hubo un error:', error);
         return of([]); // Devolvemos un array vacío como
alternativa
      })
    );
}
```

Procesando respuesta y error juntos (caso real)

```
getProductosConStock() {
  return
this.http.get<Producto[]>('https://api.ejemplo.com/productos')
    .pipe(
      map(productos => productos.filter(p => p.stock > 0)), //
procesar datos
      tap(() => console.log('Filtrando productos con stock')), //
side effect
      catchError(error => {
        console.warn('Error al cargar productos:', error);
        return of([]); // fallback seguro
      })
    );
}
```

El componente:

```
this.api.getProductosConStock()
  .subscribe(productos => {
    this.lista = productos;
  });
```

Mostrar mensajes de usuario (procesar error en UI)

```
getPost(id: number) {
  return
this.http.get<Post>(`https://api.ejemplo.com/posts/${id}`)
    .pipe(
      catchError(() => {
        return throwError(() => new Error('No se pudo cargar el
post'));
      })
    );
}
```

En un componente:

```
this.api.getPost(10).subscribe({
  next: post => this.post = post,
  error: err => this.mensaje = err.message
});
```

Retentar consultas automáticamente con retry

```
import { retry } from 'rxjs/operators';

getDatosCriticos() {
  return this.http.get('https://api.ejemplo.com/critico')
    .pipe(
      retry(3), // reintenta 3 veces antes de fallar
      catchError(() => of(null))
    );
}
```

4 Transformación de datos en el servicio de Angular

Muchas veces, el servidor nos devolverá una respuesta en formato JSON. Generalmente, los datos que queremos obtener se encontraran dentro de ese objeto JSON, o también puede que queramos hacer ciertas transformaciones intermedias como pasar las fechas de string a objeto Date, etc. Para ello usamos el método **map**.

Además de crear interfaces para cada tipo de objetos que maneje el programa (productos, usuarios, etc.), es una buena idea crear una o varias interfaces para "mapear" la respuesta del servidor (y activar el autocompletado de TypeScript):

¿Por qué transformar datos en el servicio?

- **Desacoplar** el formato de la API del **modelo de dominio** de tu app.
- **Normalizar** nombres de campos (snake_case → camelCase), tipos (string → number/Date), y estructuras (listas, árboles).
- **Simplificar** el componente: que reciba datos ya listos para pintar.

Paso 1: Crear un adaptador (transformación)

```
// Adaptador: transforma UsuarioDTO → Usuario
const adaptUsuario = (dto: any): Usuario => ({
  id: dto.id,
  firstName: dto.first_name,
  lastName: dto.last_name,
  createdAt: new Date(dto.created_at), // convertir string → Date
  fullName: `${dto.first_name} ${dto.last_name}` // propiedad
nueva
});
```

Paso 2: Usar el adaptador en el servicio

```
@Injectable({ providedIn: 'root' })
export class UsuariosService {
  private apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  getUsuarios(): Observable<Usuario[]> {
    return this.http.get<any[]>(`${this.apiUrl}/users`).pipe(
      map(dtos => dtos.map(adaptUsuario)) // ← transformación aquí
    );
  }
}
```

Paso 3: Usarlo en un componente

```
export class UsuariosComponent {
  usuarios: Usuario[] = [];

  constructor(private usuariosService: UsuariosService) {}

  ngOnInit() {
    this.usuariosService.getUsuarios().subscribe(data => {
      this.usuarios = data;
    });
  }
}
```

5 Otros métodos HTTP (get, post, put y delete)

La diferencia entre GET/DELETE y POST/PUT, es que estos últimos envían datos al servidor en el cuerpo de la petición (para insertar o modificar algo). Estos datos se envían como segundo parámetro después de la URL en la petición.

GET (Obtener datos del servidor): Se usa para **leer información** sin modificar nada en el backend.

```
getProducto(id: number): Observable<Producto> {
  return
this.http.get<Producto>(`${this.apiUrl}/productos/${id}`);
}
```

POST (Crear un nuevo recurso): Se usa cuando quieres **enviar datos al servidor para crear algo**.

```
crearProducto(producto: Producto): Observable<Producto> {
   return  this.http.post<Producto>(`${this.apiUrl}/productos`,
producto);
}
```

PUT (Actualizar un recurso existente (remplazo completo)): PUT se usa para **actualizar el recurso completo**, enviando todos los campos.

```
actualizarProducto(id:      number,      producto:      Producto):
Observable<Producto> {
  return
this.http.put<Producto>(`${this.apiUrl}/productos/${id}`,
producto);
}
```

DELETE (Eliminar un recurso del servidor): borrar un producto

```
eliminarProducto(id: number): Observable<void> {
  return
this.http.delete<void>(`${this.apiUrl}/productos/${id}`);
}
```

Un ejemplo completo de un servicio con los 4 métodos podría ser:

```
@Injectable({ providedIn: 'root' })
export class ProductosService {

  private apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  // GET
  getProductos(): Observable<Producto[]> {
    return this.http.get<Producto[]>(`${this.apiUrl}/productos`);
  }

  // GET by ID
  getProducto(id: number): Observable<Producto> {
    return this.http.get<Producto>(`${this.apiUrl}/productos/${id}`);
  }

  // POST
  crearProducto(producto: Producto): Observable<Producto> {
        return      this.http.post<Producto>(`${this.apiUrl}/productos`,
producto);
  }

  // PUT
       actualizarProducto(id:      number,      producto:      Producto):
Observable<Producto> {
     return  this.http.put<Producto>(`${this.apiUrl}/productos/${id}`,
producto);
  }

  // DELETE
  eliminarProducto(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/productos/${id}`);
  }
}
```

6 Manejar token de autenticación con angular

Las aplicaciones cliente independientes se suelen ejecutar en entornos diferentes al servidor web donde están alojados los servicios web (otro dominio, un dispositivo móvil, etc.) lo cual generalmente no permite el uso de cookies, limitadas a que el cliente se ejecute en el mismo dominio que el servidor. Por ello, la autenticación cuando trabajamos con servicios web se suele gestionar con el envío de un token al cliente, normalmente en formato JWT.

Para demostrar que estamos autenticados, nuestra aplicación cliente debe enviar en cada petición al servidor, el token recibido en el login. Normalmente en una cabecera HTTP llamada **Authorization**.

En Angular, el manejo de autenticación con token normalmente incluye:

1. **Obtener el token** desde un login (POST).
2. **Guardar el token** (generalmente en localStorage).
3. **Enviar el token automáticamente** en cada petición mediante un **HttpInterceptor**.
4. **Proteger rutas** (opcional) mediante un guard.

<u>Servicio de autenticación: Login y guardar token</u>

```
// Ejemplo de login (POST /auth/login)
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { tap, map, Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class AuthService {

  private apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  login(email: string, password: string): Observable<void> {
        return this.http.post<{ token: string
}>(`${this.apiUrl}/auth/login`, {
      email,
      password
    }).pipe(
      tap(respuesta => {
        // 2) Guardamos el token
        localStorage.setItem('token', respuesta.token);
      }),
      map(() => void 0)
    );
```

```
  }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  logout(): void {
    localStorage.removeItem('token');
  }
}
```

HttpInterceptor: Añadir el token automáticamente

```
import { Injectable } from '@angular/core';
import {
  HttpInterceptor, HttpRequest, HttpHandler, HttpEvent
} from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from './auth.service';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {

  constructor(private authService: AuthService) {}

  intercept(
    request: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>> {

    const token = this.authService.getToken();

    if (!token) {
      // Sin token → continuar sin modificar petición
      return next.handle(request);
    }

    // Clonamos la petición para añadir el header Authorization
    const requestWithToken = request.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });

    return next.handle(requestWithToken);
  }
}
```

Registrar el interceptor en Angular (En Angular 15–20, el registro se hace desde el *provider* del módulo o del bootstrapApplication.)

```
import { HTTP_INTERCEPTORS } from '@angular/common/http';

@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true
    }
  ]
})
export class AppModule {}
```

Consumir login desde un componente

```
import { Component } from '@angular/core';
import { AuthService } from '../auth.service';

@Component({
  selector: 'app-login',
  template: `
    <h2>Login</h2>

    <input [(ngModel)]="email" placeholder="Email">
        <input      [(ngModel)]="password"      placeholder="Password"
type="password">

    <button (click)="iniciarSesion()">Entrar</button>
  `
})
export class LoginComponent {

  email = '';
  password = '';

  constructor(private auth: AuthService) {}

  iniciarSesion() {
    this.auth.login(this.email, this.password).subscribe({
      next: () => console.log('Login correcto'),
      error: err => console.warn('Error en login:', err)
    });
  }
}
```

Servicio con una llamada protegida (NO le pasas token a mano)

```
@Injectable({ providedIn: 'root' })
export class PerfilService {

  private apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  getPerfil(): Observable<Usuario> {
    // 4) Llamada protegida → El interceptor mete el token
    return this.http.get<Usuario>(`${this.apiUrl}/mi-perfil`);
  }
}
```

Consumo real: el token viaja automáticamente

```
this.perfilService.getPerfil().subscribe(perfil => {
  console.log('Datos del usuario:', perfil);
});
```

7 Usando un interceptor para enviar el token

Los Interceptores son servicios que "interceptan" todas las peticiones HTTP que realizamos de forma automática y permiten alterar tanto la petición (antes de enviarla al servidor),como la respuesta. Un Interceptor se crea como un servicio de Angular que implementa la interfaz HttpInterceptor. Esto nos obliga a crear un método llamado **intercept.**

Servicio de autenticación (fuente del token): Un servicio mínimo que guarda/lee el token (ej. en localStorage)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { tap, map, Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  login(email: string, password: string): Observable<void> {
    return this.http.post<{ token: string }>(`${this.apiUrl}/auth/login`, { email, password }).pipe(
      tap(r => localStorage.setItem('token', r.token)),
      map(() => void 0)
```

```
    );
  }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  logout(): void {
    localStorage.removeItem('token');
  }
}
```

Interceptor que añade el token: Versión básica (añade Authorization si hay token)

```
import { Injectable } from '@angular/core';
import {
  HttpInterceptor, HttpRequest, HttpHandler, HttpEvent, HttpResponse
} from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from './auth.service';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {

  constructor(private auth: AuthService) {}

      intercept(req:      HttpRequest<any>,      next:      HttpHandler):
Observable<HttpEvent<any>> {
    // Evitar añadir token a endpoints públicos (ej.: /auth/login)
       const    isAuthEndpoint    =    req.url.includes('/auth/login')    ||
req.url.includes('/auth/register');

    const token = this.auth.getToken();
    if (!token || isAuthEndpoint) {
      return next.handle(req); // sin cambios
    }

    // Clonar petición con header Authorization
    const reqWithToken = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });

    return next.handle(reqWithToken);
  }
}
```

Hay variantes útiles

Añadir también **Content-Type / Accept** solo si no existen

```
const headers = req.headers
  .set('Accept', 'application/json')
      .set('Content-Type',     req.headers.get('Content-Type')     ??
'application/json');

const reqWithHeaders = req.clone({ headers, setHeaders: {
Authorization: `Bearer ${token}` } });
```

Excluir **dominios externos** (no controlas CORS ni autentican igual)

```
const                                isExternal                                =
!req.url.startsWith('https://api.ejemplo.com');
if (isExternal) return next.handle(req);
```

Registro del interceptor

Opción A — Proyectos con AppModule (HttpClientModule)

```
import { NgModule } from '@angular/core';
import  {  HTTP_INTERCEPTORS,  HttpClientModule  }  from
'@angular/common/http';

@NgModule({
  imports: [HttpClientModule],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor,
multi: true }
  ]
})
export class AppModule {}
```

Opción B — Bootstrap standalone (Angular moderno). Si usas bootstrapApplication y provideHttpClient:

```
import  {  bootstrapApplication  }  from  '@angular/platform-
browser';
import  {  provideHttpClient,  withInterceptorsFromDi  }  from
'@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(withInterceptorsFromDi()),
```

```
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor,
multi: true }
  ]
});
```

Uso en servicios (no se escribe el header a mano)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class PerfilService {
  private readonly apiUrl = 'https://api.ejemplo.com';

  constructor(private http: HttpClient) {}

  getPerfil(): Observable<Usuario> {
    // El interceptor añadirá el token automáticamente
    return this.http.get<Usuario>(`${this.apiUrl}/mi-perfil`);
  }
}
```

Manejo de errores 401 (opcional pero recomendable): Si el backend responde **401 Unauthorized**, puedes: cerrar sesión, redirigir a login, o intentar refresh.

```
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

      intercept(req:      HttpRequest<any>,      next:      HttpHandler):
Observable<HttpEvent<any>> {
    const token = this.auth.getToken();
    const isAuthEndpoint = req.url.includes('/auth/');

    const request = (!token || isAuthEndpoint) ? req : req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });

    return next.handle(request).pipe(
      catchError(err => {
        if (err.status === 401) {
          // Token inválido/expirado → limpiar sesión y (opcional) redirigir
          this.auth.logout();
```

```
        // router.navigate(['/login']); // si inyectas Router
      }
      return throwError(() => err);
    })
  );
 }
}
```

8 Usando un interceptor para la URL base

En lugar de escribir this.http.get('https://api.midominio.com/usuarios'); uedes usar un **interceptor** que **preprenda automáticamente** la URL base https://api.midominio.com de forma que tú solo escribas this.http.get('/usuarios');

Esto:

- Reduce repetición de código
- Evita errores en la URL
- Permite cambiar la API desde un único lugar
- Hace mucho más limpio tu código en servicios

1) Crear el interceptor de URL base: Este interceptor detecta si la URL **no es absoluta** y entonces le añade la URL base:

```
import { Injectable } from '@angular/core';
import {
  HttpInterceptor, HttpRequest, HttpHandler, HttpEvent
} from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class BaseUrlInterceptor implements HttpInterceptor {

  private readonly baseUrl = 'https://api.midominio.com';

      intercept(req:      HttpRequest<any>,      next:      HttpHandler):
Observable<HttpEvent<any>> {

    // Si la URL ya es absoluta (http/https), no la tocamos
    if (req.url.startsWith('http://') || req.url.startsWith('https://')) {
      return next.handle(req);
    }

    // Crear la nueva URL completa
    const apiReq = req.clone({
      url: `${this.baseUrl}${req.url}`
```

```
    });

    return next.handle(apiReq);
  }
}
```

Registrar el interceptor

En AppModule:

```
import { NgModule } from '@angular/core';
import { HTTP_INTERCEPTORS } from '@angular/common/http';

@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: BaseUrlInterceptor,
      multi: true
    }
  ]
})
export class AppModule {}
```

O con bootstrapApplication (Angular standalone):

```
import { HTTP_INTERCEPTORS, provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(withInterceptorsFromDi()),
    {
      provide: HTTP_INTERCEPTORS,
      useClass: BaseUrlInterceptor,
      multi: true
    }
  ]
});
```

Servicios limpiísimos sin URL base

Ahora tus servicios quedan así:

```
@Injectable({ providedIn: 'root' })
export class UsuariosService {
  constructor(private http: HttpClient) {}

  getUsuarios() {
```

```
    return this.http.get('/usuarios'); // el interceptor añade la base URL
  }

  getUsuario(id: number) {
    return this.http.get(/usuarios/${id});
  }
}
```